

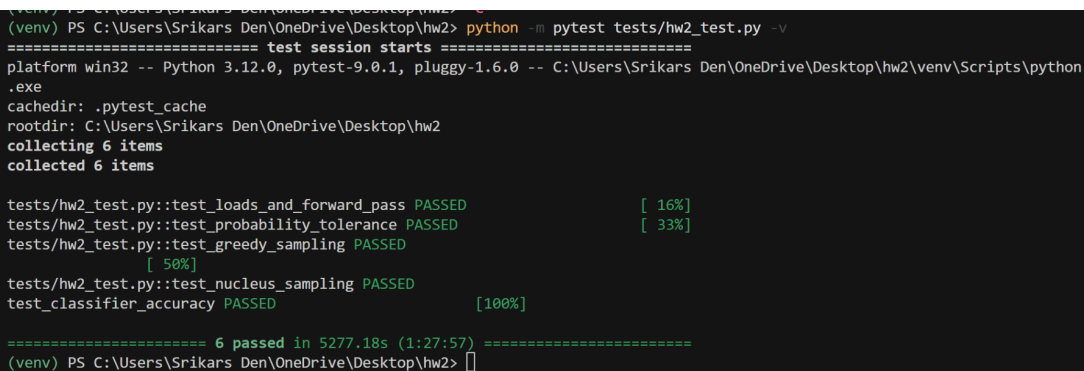
Implementation Status

I successfully completed all parts of the assignment, including implementing the GPT-2 architecture from scratch, the generation function with greedy and nucleus sampling, KV caching, and the sequence classification model.

For Part 1, I implemented the full GPT-2 forward pass, attention mechanism, and generation pipeline. This included correctly handling positional embeddings, causal masking, and efficient decoding using KV caching.

For Part 2, I implemented GPTSequenceClassification, reusing the GPT-2 backbone and adding a classification head. I also implemented the full training pipeline, including dataset loading, batching, padding, training loop, evaluation, and checkpoint saving.

All visible test cases passed successfully. Here's a screenshot of the results after running all the test cases:



```
(venv) PS C:\Users\Srikars Den\OneDrive\Desktop\hw2> python -m pytest tests/hw2_test.py -v
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.0.1, pluggy-1.6.0 -- C:\Users\Srikars Den\OneDrive\Desktop\hw2\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Srikars Den\OneDrive\Desktop\hw2
collecting 6 items
collected 6 items

tests/hw2_test.py::test_loads_and_forward_pass PASSED [ 16%]
tests/hw2_test.py::test_probability_tolerance PASSED [ 33%]
tests/hw2_test.py::test_greedy_sampling PASSED [ 50%]
tests/hw2_test.py::test_nucleus_sampling PASSED
test_classifier_accuracy PASSED [100%]

===== 6 passed in 5277.18s (1:27:57) =====
(venv) PS C:\Users\Srikars Den\OneDrive\Desktop\hw2>
```

Training Techniques, Loss, and Validation Accuracy Curves

To exceed the required 65% validation accuracy, I used the following techniques:

- Initialized the classifier using pretrained GPT-2 language model weights
- Used left padding to ensure the last token representation remained meaningful
- Used the last token representation ($x[:, -1, :]$) as the sequence embedding
- Optimized using AdamW with learning rate $2e-5$
- Applied gradient clipping to stabilize training
- Trained for 3 epochs with batch size 8 and max sequence length 256

Training was performed on a GPU (Tesla T4 via Google Colab), which significantly improved runtime.

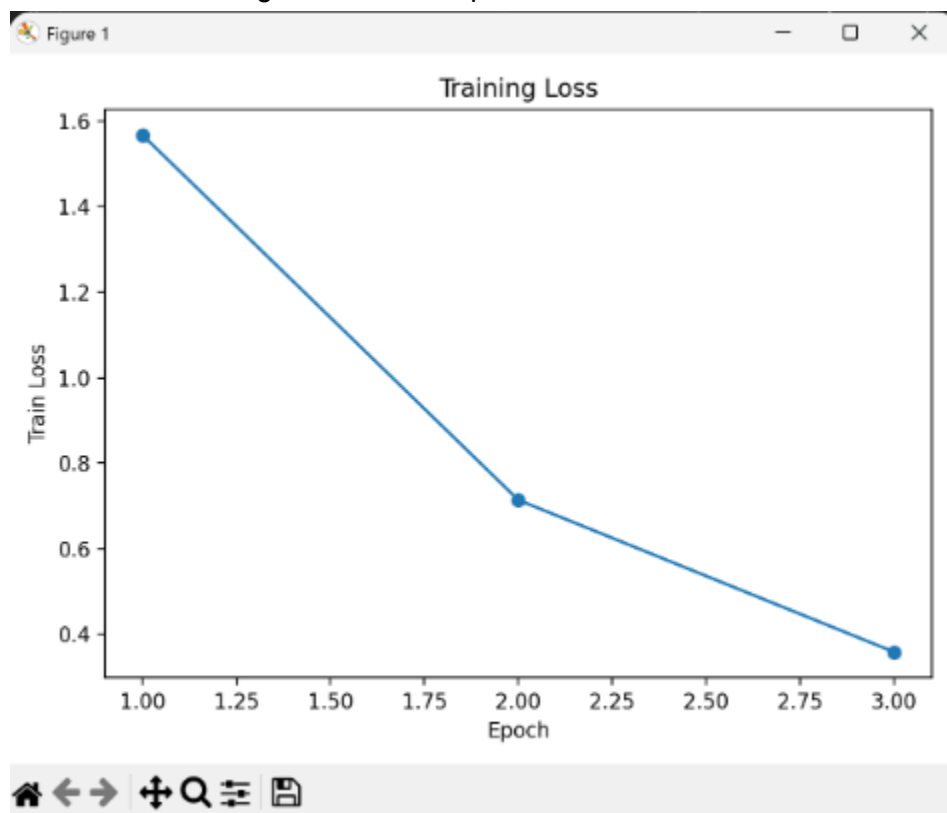
Results

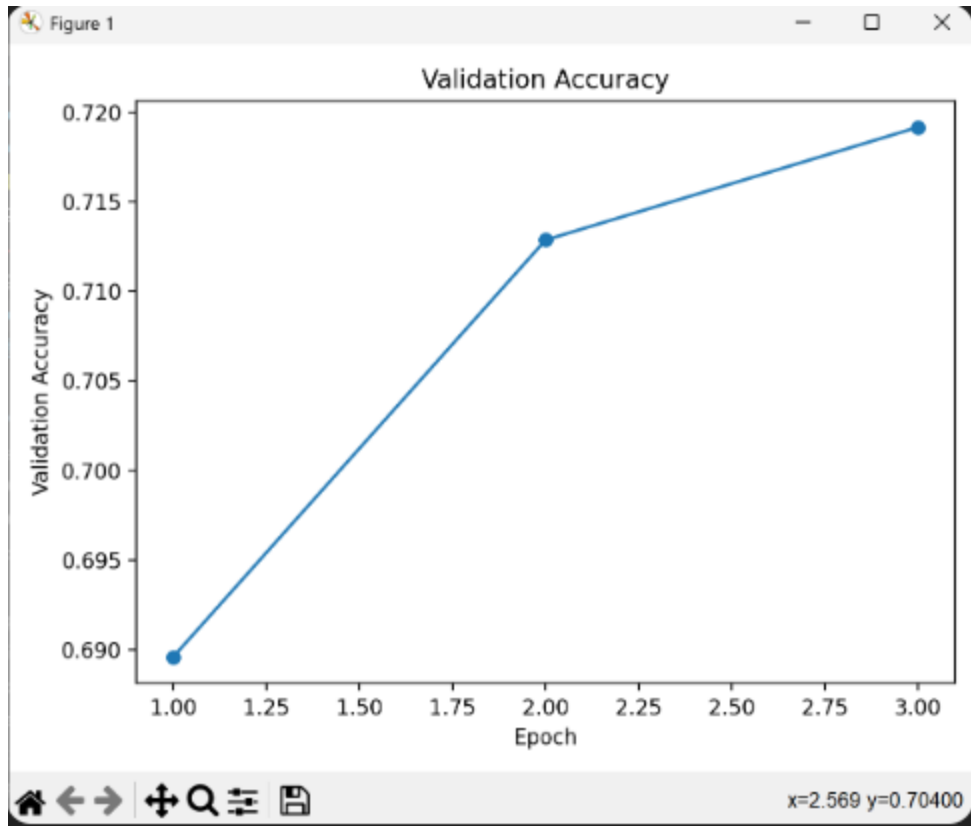
- Final validation accuracy: **71.92%**
- Training loss decreased steadily across epochs
- Validation accuracy improved consistently across epochs

Observations

- Training loss decreased from $\sim 1.56 \rightarrow \sim 0.36$
- Validation accuracy improved from $\sim 0.69 \rightarrow \sim 0.72$
- Validation loss slightly increased in the final epoch, suggesting mild overfitting, but accuracy continued improving

Here are the training and validation plots:





Challenges and Solutions

One of the main challenges was ensuring that the classifier architecture aligned correctly with the input batching strategy.

Initially, it was unclear how to extract a meaningful sequence representation from GPT-2 outputs. I resolved this by using the last token representation ($x[:, -1, :]$), which captures the full context due to causal attention. This required switching to left padding, since right padding would cause the final token to be a padding token rather than real input.

Another challenge was debugging environment issues in Google Colab. When running the test suite, I encountered errors related to torchvision and transformers. This was resolved by uninstalling torchvision, which allowed the Hugging Face imports to work correctly.

Training performance was also initially a concern when running locally on CPU, as it was very slow. Switching to GPU training in Colab significantly improved runtime and allowed for efficient experimentation.

Overall, the main lessons learned were:

- the importance of aligning model architecture with data preprocessing
- how padding strategy affects model behavior
- how pretrained models can be effectively fine-tuned for downstream tasks
- how to debug both code-level and environment-level issues

AI Usage and Collaboration

I used AI tools (primarily ChatGPT) throughout this assignment to assist with both implementation and understanding.

AI was used for:

- clarifying conceptual topics such as attention, KV caching, and training workflows
- debugging errors and environment issues
- structuring parts of the implementation, such as the training loop and dataset pipeline
- explaining PyTorch operations and tensor shapes

However, I made sure to actively understand the logic behind each component rather than blindly copying code. I iterated on implementations, debugged issues myself, and verified correctness through testing and training results.